

Week 5 | Uninformed Search Strategies (BFS, DFS, UCS)

CLO - 2: Apply classical AI techniques, including uninformed and informed search algorithms, minimax, and neural networks, to solve structured problems.

Note: In all tasks, visualize each graph with the solution path highlight using **networkX** Library.

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the Single .ipynb file** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the total marks.**

Task 1: Breadth-First Search (BFS) for Minimum-Transfer Public Transport Routing

Objective

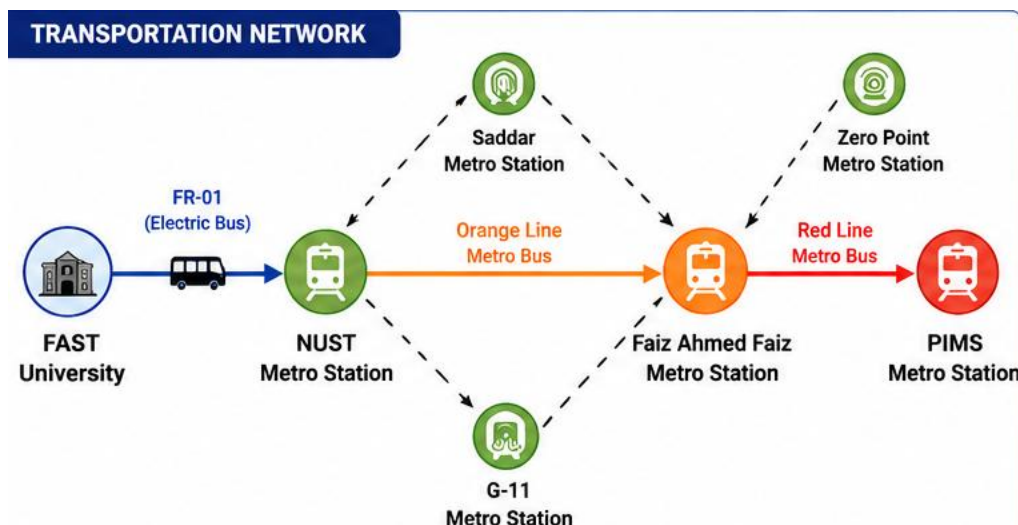
Implement Breadth-First Search to find the route requiring the minimum number of transitions from FAST University to PIMS Metro Station.

Problem Description

A student needs to travel from **FAST University Islamabad** to the **PIMS Metro Station / Centaurus-PIMS area** using the Islamabad public transport network.

For this task, consider each connection between two stations as **one equal-cost transition**, regardless of the actual distance or travel time.

Use the following transportation network:



The important route represented in the network is:

FAST University → NUST Metro Station → Faiz Ahmed Faiz Metro Station → PIMS Metro Station

This requires **3 transitions**.

The graph also contains alternative connections so that **BFS** must explore multiple branches instead of simply following a predefined route.

Initial State: FAST University

Goal State: PIMS Metro Station

Implementation Requirements

- Represent the transportation network using an adjacency list.
- Use a **FIFO queue** for frontier management.
- Maintain a **visited/explored set**.
- Maintain a parent dictionary for path reconstruction (solution path).
- Record and display the **order in which stations are expanded**.
- Display the **frontier/queue after each major step**.
- Record the **level/depth of each station**.
- Apply the goal test when a node is removed from the queue.
- Terminate search immediately upon goal detection
- Print:
 - Traversal/expansion order
 - Level of each explored station
 - Final route
 - Number of transitions

Expected Computational Behavior

Your implementation should demonstrate that:

- BFS explores the network **level by level**.
- Stations directly reachable from FAST are explored before stations two transitions away.

- The first discovered goal corresponds to the route with the **minimum number of transitions** in this unweighted graph. (Optimal with respect to path length (number of edges)).

Required Output

Your output should be similar to:

Initial State: FAST University

Goal State: PIMS Metro Station

Level 0: FAST University

Level 1: NUST Metro Station

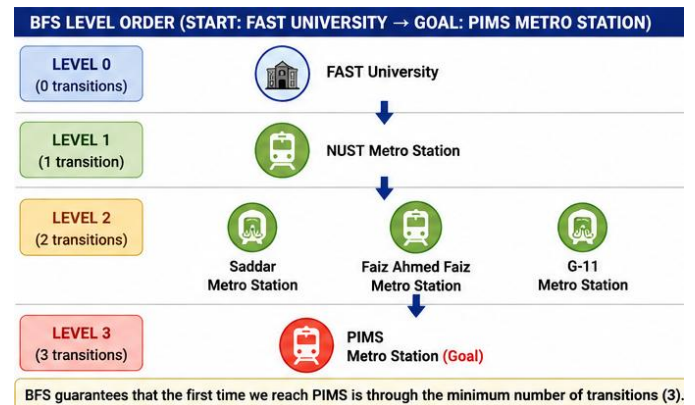
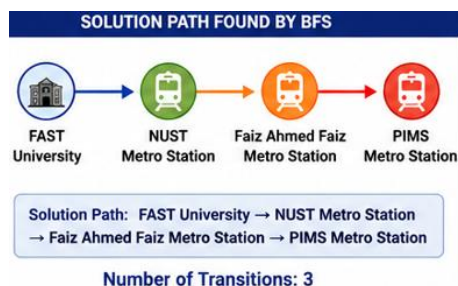
Level 2: Faiz Ahmed Faiz Metro Station

Level 3: PIMS Metro Station

Traversal Order:

...

Solution Path:



Important: Do not hard code the final route. The route must be generated by your BFS implementation.

Visualization Requirement

Visualize the transportation network using the networkx library.

Clearly distinguish:

- Initial state
- Goal state
- Intermediate stations
- Connections between stations

Task 2: Tree Search vs Graph Search: Smart Building Navigation

Objective

Understand the fundamental difference between **Tree Search** and **Graph Search** by implementing BFS on:

1. A **tree representation** of a building navigation problem
2. A **graph representation** of the same environment containing a cycle
3. The graph representation with a **visited set**

The task demonstrates why real-world environments are generally represented as graphs and why graph-search algorithms must keep track of previously explored states.

Problem Description

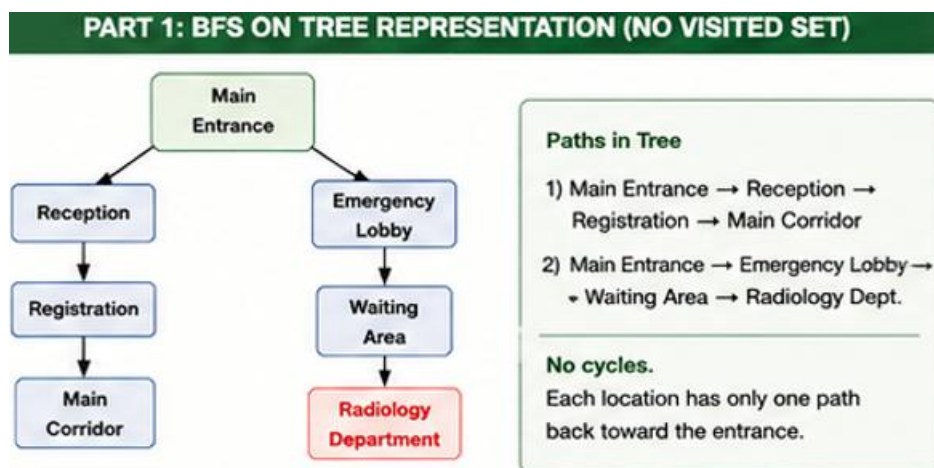
Consider an AI-powered indoor navigation system helping a visitor navigate through a **hospital building**.

The visitor starts at the: **Main Entrance** and needs to reach: **Radiology Department**

The hospital contains several connected areas. Initially, suppose the navigation system represents the building as a **tree**, where every location has only one route back toward the entrance.

Part 1: Hospital Navigation as a Tree (BFS on Tree Representation)

The simplified tree representation is:



- There are **no connections between branches**.
- This is a **tree structure** because there are no back-edges creating cycles.

Implement BFS on the above hospital navigation tree.

Initial State: Main Entrance

Goal State: Radiology Department

Implementation Requirements

- Use a FIFO queue.
- **Do NOT use a visited set** in this part.
- Record the node expansion order.
- Display the queue after each step.
- Use parent pointers to reconstruct the final path.
- Stop when the goal is found.

Expected Queue Behavior

Your implementation should produce behavior similar to:

Step 1: Take → Main Entrance

Add: Reception, Emergency Lobby

Queue: [Reception, Emergency Lobby]

Then:

Step 2: Take → Reception

Add: Registration

Queue: [Emergency Lobby, Registration]

Then:

Step 3: Take → Emergency Lobby

Add: Waiting Area

Queue: [Registration, Waiting Area]

Continue until: Radiology Department is reached.

What You Observe

The search finishes normally because:

- No location points back to an already explored location.
- There are no cycles.
- The queue eventually becomes empty if the goal is not found.

- Every location can be processed without repeatedly generating the same state.

Expected Output

Traversal Order: Main Entrance → Reception → Emergency Lobby → Registration → Waiting Area → Radiology Department (**Goal Found!**)

Path: Main Entrance → Emergency Lobby → Waiting Area → Radiology Department

Part 2: Hospital Navigation as a Graph

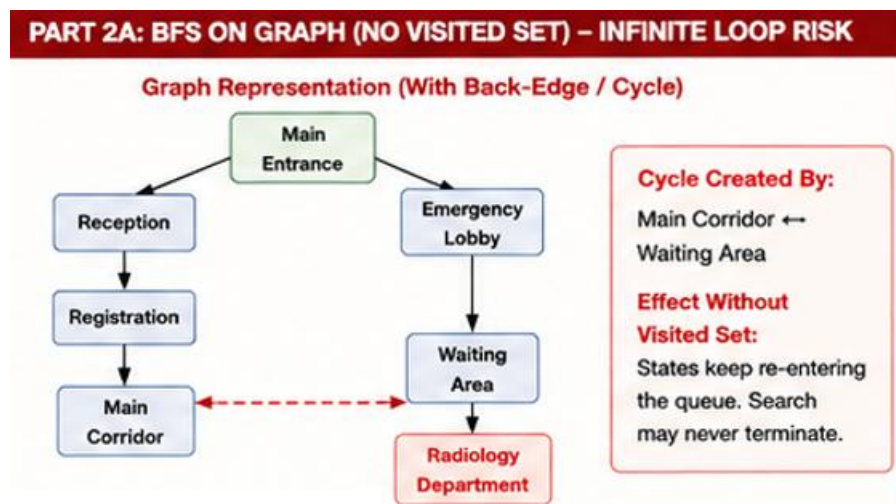
In a real hospital, however, different corridors may connect to each other. For example, suppose there is an additional corridor connecting:

Main Corridor to: Waiting Area

and another connection between: Waiting Area and: Main Corridor

Now the environment is no longer a tree. It is a **graph**.

The graph representation becomes:



The important additional connection is:

Main Corridor ↔ Waiting Area

This creates a cycle.

More directly, depending on the exact adjacency representation, the important point is that a previously reachable state can be reached again through another branch.

Notice the difference:

Tree: Each location belongs to one branch.

Graph: A location can have connections to locations already reachable through another branch.

Part 2A: BFS on Graph WITHOUT a Visited Set

Run the **same BFS implementation**, but this time: **Do NOT use a visited set**.

Search for: Radiology Department

Implementation Requirements

Display:

Step	Current Node	Nodes Added	Queue
1			
2			
3			
4			
...			

Continue observing the queue.

What you Should Look For

At some point, the search reaches:

Main Corridor and generates: Waiting Area

Later, the search reaches: Waiting Area which generates: Main Corridor again.

Therefore:

Main Corridor

↓

Waiting Area

↓

Main Corridor

↓

Waiting Area

↓

Main Corridor

↓

...

The same states continue entering the frontier.

Expected Behavior

The queue may begin to contain repeated locations:

[Waiting Area, Main Corridor]

then:

[Main Corridor, Waiting Area, ...]

and continue growing.

The search can therefore:

Keep revisiting states

↓

Generate the same states again

↓

Never properly terminate

if the implementation does not impose a stopping mechanism.

Part 2B: BFS on Graph WITH a Visited Set

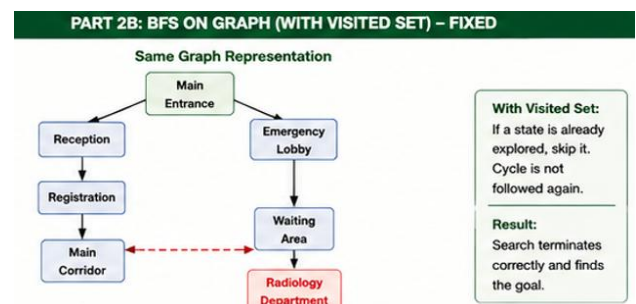
Now modify the implementation by adding:

visited = set()

Run BFS on the **same hospital graph**.

Implementation Requirements

- Maintain the FIFO queue.
- Maintain a visited set.
- Before expanding a node, check whether it has already been visited.
- Add a node to the visited set when it is processed.
- Skip already explored states.
- Record the queue and visited set after each step.



- Reconstruct the final path using parent pointers.

Expected Execution

Suppose the search reaches:

Main Corridor and explores: Waiting Area

Later, Waiting Area attempts to generate: Main Corridor

But: "Main Corridor" in visited is: True

Therefore:

Main Corridor → already visited → **SKIP**

The cycle is no longer repeatedly explored. The search can continue toward: Radiology Department and terminate normally.

What You Observe

The important difference is:

Tree Search

No cycles

↓

No repeated states

↓

Visited set is not essential for preventing cycles

Graph Search

Multiple connections

↓

Cycles / repeated states possible

↓

Same state may be generated again

↓

Visited set required

Tree vs Graph Representation

Complete the following comparison:

Feature	Tree	Graph
Multiple paths to same location	No	Yes
Back-edges	No	Possible
Cycles	No	Possible
Repeated states possible	No	Yes
BFS without visited set	Can work	Can fail/infinite loop
Visited set	Not required for cycle prevention	Essential
Real-world example	Hierarchical representation	Actual interconnected building

Part 3: Compare the Three Implementations

You have now implemented BFS in three configurations:

Case 1: BFS + Tree + No Visited Set

Case 2: BFS + Graph + No Visited Set

Case 3: BFS + Graph + Visited Set

Complete the following table:

Configuration	Cycles?	Visited Set?	Repeated States?	Terminates Correctly?
Tree Search	No	No	No	Yes
Graph Search	Yes/Possible	No	Yes	May not
Graph Search	Yes/Possible	Yes	Prevented	Yes

Tree Search assumes that the search space has no repeated states through different paths, whereas Graph Search explicitly handles the possibility of reaching the same state multiple times. The visited/explored set prevents repeated expansion and makes search practical on cyclic real-world environments.

Task 3: Depth-First Search (DFS) for Warehouse Robot Exploration

Objective

Implement Depth-First Search to explore a hierarchical environment by going **as deep as possible before backtracking**.

Problem Description

Consider an autonomous warehouse robot operating inside a distribution center.

The warehouse is divided into several connected zones:

The robot starts from: Receiving Area

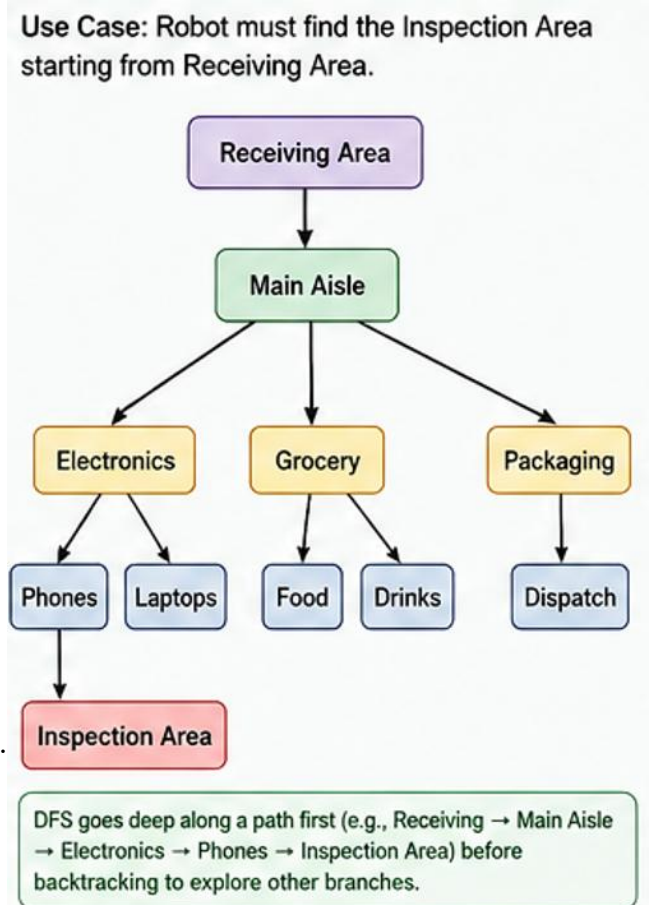
and must locate: Inspection Area

Initial State: Receiving Area

Goal State: Inspection Area

Implementation Requirements

- Implement DFS using either:
 - An explicit **LIFO stack**, or
 - Recursion.
- Maintain a visited set.
- Record the node expansion order.
- Display the stack after each major step.
- Use parent pointers to reconstruct the final route.
- Print:
 - Traversal order
 - Solution path
 - Number of states explored



Expected Computational Behavior

DFS should demonstrate:

Select a branch

↓

Explore deeper



Reach a dead end / completed branch



Backtrack



Explore another branch

*The robot should **not** be modeled as simply checking all immediate neighboring zones before going deeper.*

Important Observation

The exact DFS traversal can depend on the order in which neighbors are stored.

Therefore, you must report: Traversal Order, separately from: Solution Path

Task 4: Uniform Cost Search (UCS) for Delivery Route Optimization

Objective

Implement Uniform Cost Search to find the **minimum-cost route** for an autonomous delivery vehicle.

Problem Description

An AI-powered delivery system needs to transport a parcel from:

Islamabad Delivery Hub

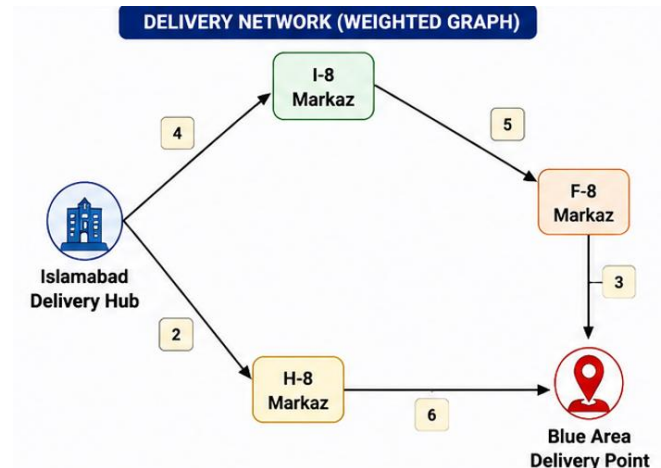
to:

Blue Area Delivery Point

Different road segments have different estimated costs because of factors such as:

- Distance
- Expected travel time
- Road conditions
- Delivery restrictions

For this lab, the following **illustrative costs** are assigned to the road segments.



Initial State: Islamabad Delivery Hub

Goal State: Blue Area Delivery Point

*Note: The values above are **illustrative laboratory costs**, not live travel-time estimates.*

Implementation Requirements

- Use Python's **heapq** priority queue.
- Store: (total_cost, current_state)
- Expand the state having the **lowest cumulative path cost**.
- Maintain the best-known cost for each state.
- Use parent pointers.
- Apply the goal test when the node is removed from the priority queue.
- Print:
 - Current node
 - Cumulative cost
 - Priority queue contents
 - Traversal order
 - Final route
 - Total path cost

Expected Computational Behavior

You should demonstrate that UCS does **not** simply select the route with: Fewest edges

Instead, it selects: Lowest accumulated path cost

For example:

Cost so far $\rightarrow$ Cost of previous path + Cost of new edge $\rightarrow$ New cumulative cost

UNGRADED HOME TASK:

Task 5: Formal State-Space Modeling for an Indoor Service Robot

Objective

Construct a formal state-space representation of a realistic AI navigation problem before applying a search algorithm.

Problem Description

An indoor service robot operates in a hospital and needs to deliver a medical package from the:

Pharmacy to the: Emergency Department

The robot can move between connected hospital areas.

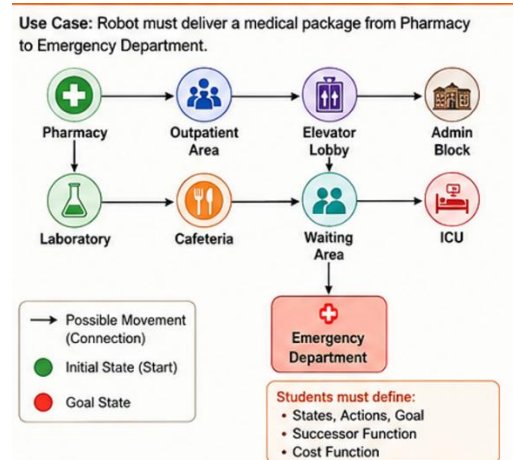
Represent the problem as a directed graph.

You Must Clearly Define

- Initial State
- Goal State
- State Representation
- Possible Actions
- Successor Function
- Cost Function
- Path

Implementation Requirements

1. Represent the environment using an adjacency list.
2. Create meaningful real-world state names.
3. **Separate the problem definition from the search algorithm.**
4. Implement a function that returns the successors of a given state.
5. Define an appropriate cost representation.



6. Demonstrate how the same problem representation could be used by:

- BFS
- DFS
- UCS

Required Output

Prepare a short table:

Component	Your Definition
Initial State	
Goal State	
States	
Actions	
Successor Function	
Cost Function	

Expected Outcome

You should understand that **problem formulation comes before search algorithm implementation.**

The search algorithm should operate on the problem representation rather than requiring the graph to be redesigned for every algorithm.

Task 6: Smart Building Navigation Agent

Objective

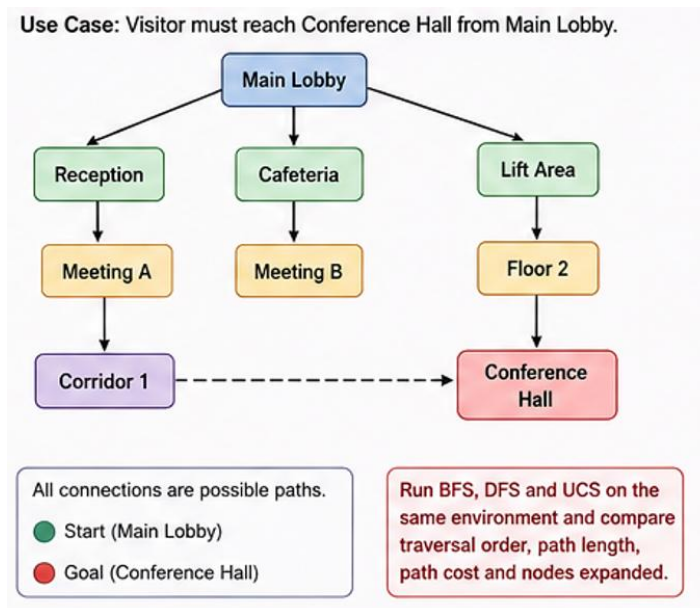
Apply uninformed search techniques to a realistic navigation problem and compare how different search strategies behave on the **same environment.**

Problem Description

A smart-building navigation agent is helping a visitor navigate through a large office complex.

The visitor enters through: **Main Lobby** and needs to reach: **Conference Hall**

The building contains multiple connected areas:



Initial State: Main Lobby

Goal State: Conference Hall

Implementation Requirements

Implement all three algorithms on the same environment:

BFS

Use BFS to determine: Minimum number of transitions

DFS

Use DFS to: Explore the building hierarchy deeply and observe where backtracking occurs.

UCS

Assign illustrative costs to building transitions based on: Walking distance / estimated movement cost and determine: Minimum-cost route

You Must Record

For each algorithm:

1. Traversal order
2. Solution path
3. Number of nodes expanded
4. Path length
5. Path cost

6. Frontier behavior

7. Whether the returned path is optimal under the given objective

*The purpose of this task is **not** to produce three identical routes. You should analyze why the algorithms may behave differently even though the underlying environment is the same.*

Students should be able to connect the implementation results to the theoretical properties of the algorithms rather than simply reporting which algorithm "worked."

Visualization Requirement

In all tasks, visualize the graph using the networkx library.

The visualization should clearly show:

- Nodes/states
- Connections/edges
- Edge weights where applicable
- Initial state
- Goal state

For weighted graphs, display the edge cost on the graph.

For Task 1, clearly label the transportation services where appropriate:

FR-01 (Electric Bus)

Orange Line Metro Bus

Red Line Metro Bus

The transportation graph should reflect the provided route:

FAST University

↓

NUST Metro Station

↓

Faiz Ahmed Faiz Metro Station

↓

PIMS Metro Station